#imbalanced dataset

Dataset yang tidak seimbang (imbalanced dataset) merujuk pada situasi di mana jumlah sampel dalam setiap kelas atau kategori tidak seimbang secara signifikan. Ini sering terjadi dalam masalah klasifikasi di mana satu kelas memiliki jumlah sampel yang jauh lebih banyak daripada kelas lainnya. Misalnya, dalam masalah deteksi penipuan kartu kredit, jumlah transaksi penipuan mungkin sangat sedikit dibandingkan dengan jumlah transaksi normal.

Keberadaan dataset yang tidak seimbang dapat menyebabkan beberapa masalah saat menggunakan algoritma machine learning. Algoritma cenderung menjadi bias terhadap kelas mayoritas, yang dapat mengakibatkan kelas minoritas diabaikan atau diklasifikasikan dengan akurasi yang rendah. Ini disebabkan oleh fakta bahwa algoritma cenderung mempelajari pola dari kelas mayoritas dengan lebih baik daripada kelas minoritas.

Untuk menangani dataset yang tidak seimbang, beberapa pendekatan yang umum digunakan adalah:

Oversampling: Menambahkan lebih banyak sampel dari kelas minoritas untuk membuatnya seimbang dengan kelas mayoritas. Undersampling: Mengurangi jumlah sampel dari kelas mayoritas untuk membuatnya seimbang dengan kelas minoritas. Synthetic Minority Over-sampling Technique (SMOTE): Membuat sampel sintetis untuk kelas minoritas dengan teknik interpolasi.

Tugas 1

- Download dataset : https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset (https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset)
- Upload dataset ke gdrive atau github masing-masing
- Tampilkan dataset dalam dataframe
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dari dataset ini

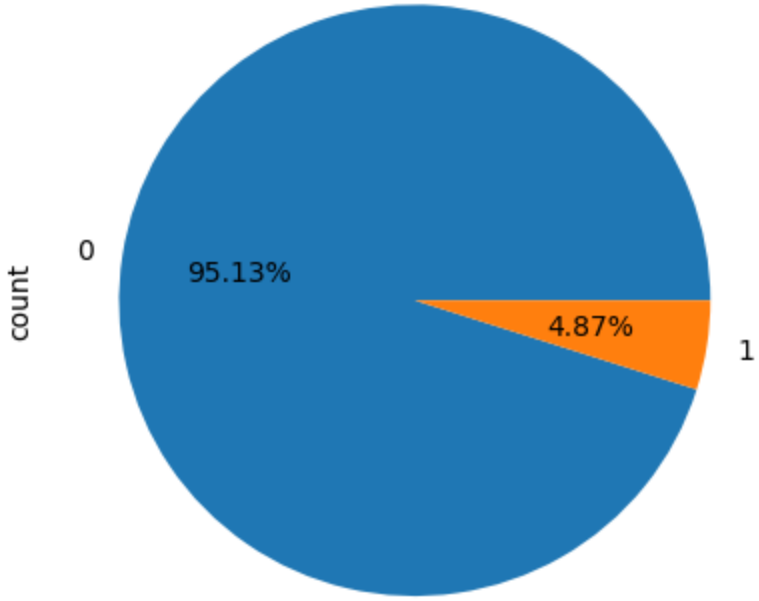
In [1]:
```
import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/healthcare-dataset-stroke-data.
df.head()
```

Out[1]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

In [2]:
```
import matplotlib.pyplot as plt

data = df['stroke'].value_counts()
data.plot(kind='pie',autopct='%.2f%%')
plt.show()
```



In [3]:
```
df.isnull().sum()
```

Out[3]:
```
id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                201
smoking_status       0
stroke               0
dtype: int64
```

random under sampling

In [4]:
```
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

under_sampling = RandomUnderSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df[['stroke']]

le = LabelEncoder()
df_y = le.fit_transform(df_y)

#imputation
df_X['bmi'].fillna(df_X['bmi'].mean(), inplace=True)

#categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25, random_state=42)

X_resampled, y_resampled = under_sampling.fit_resample(X_train, y_train)

print('sebelum undersampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah undersampling')
print(pd.DataFrame(y_resampled).value_counts())
```

```
sebelum undersampling
0    3663
1     169
Name: count, dtype: int64
setelah undersampling
0    169
1    169
Name: count, dtype: int64
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:116: DataConversionWarning: A column-vector y
was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

In [5]:
```
df_X
```

Out[5]:

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status
0	1	67.0	0	1	1	2	1	228.69	36.600000	1
1	0	61.0	0	0	1	3	0	202.21	28.893237	2
2	1	80.0	0	1	1	2	0	105.92	32.500000	2
3	0	49.0	0	0	1	2	1	171.23	34.400000	3
4	0	79.0	1	0	1	3	0	174.12	24.000000	2
...	...	...	...	...	...	...	...	...	...	...
5105	0	80.0	1	0	1	2	1	83.75	28.893237	2
5106	0	81.0	0	0	1	3	1	125.20	40.000000	2
5107	0	35.0	0	0	1	3	0	82.99	30.600000	2
5108	1	51.0	0	0	1	2	0	166.29	25.600000	1
5109	0	44.0	0	0	1	0	1	85.28	26.200000	0

5110 rows × 10 columns

In [6]:
```
df_y = df[['stroke']]
df_y
```

Out[6]:

	stroke
0	1
1	1
2	1
3	1
4	1
...	...
5105	0
5106	0
5107	0
5108	0
5109	0

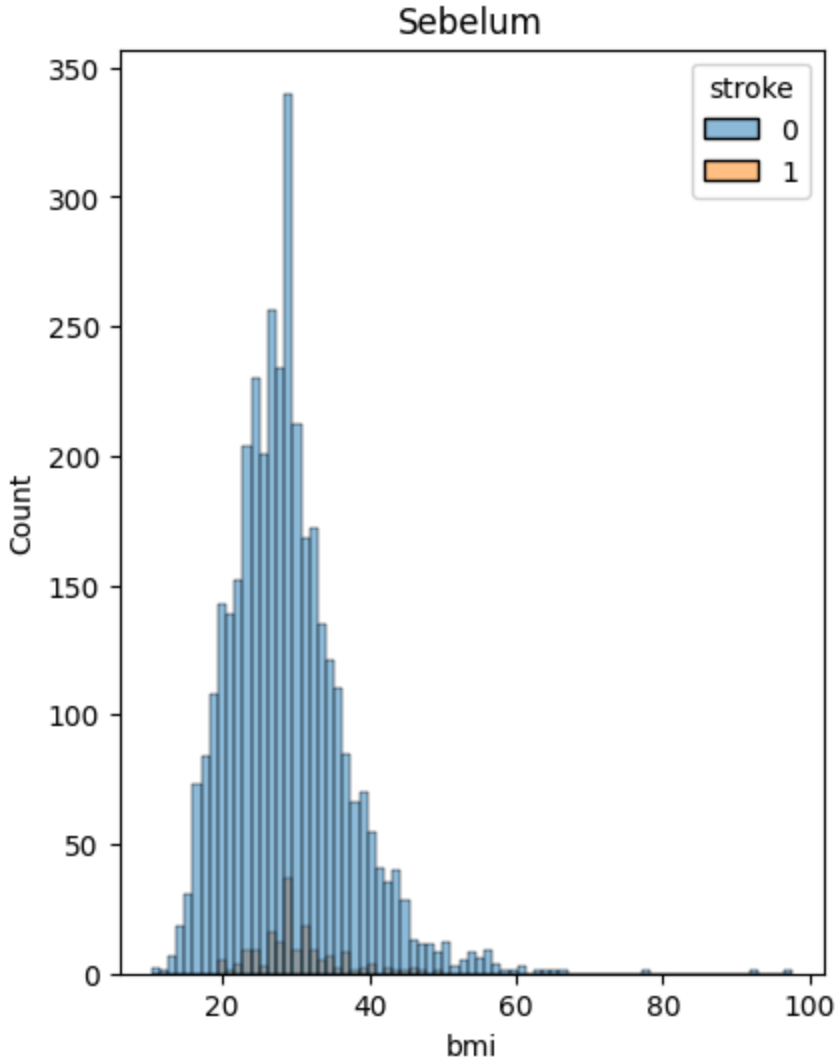
5110 rows × 1 columns

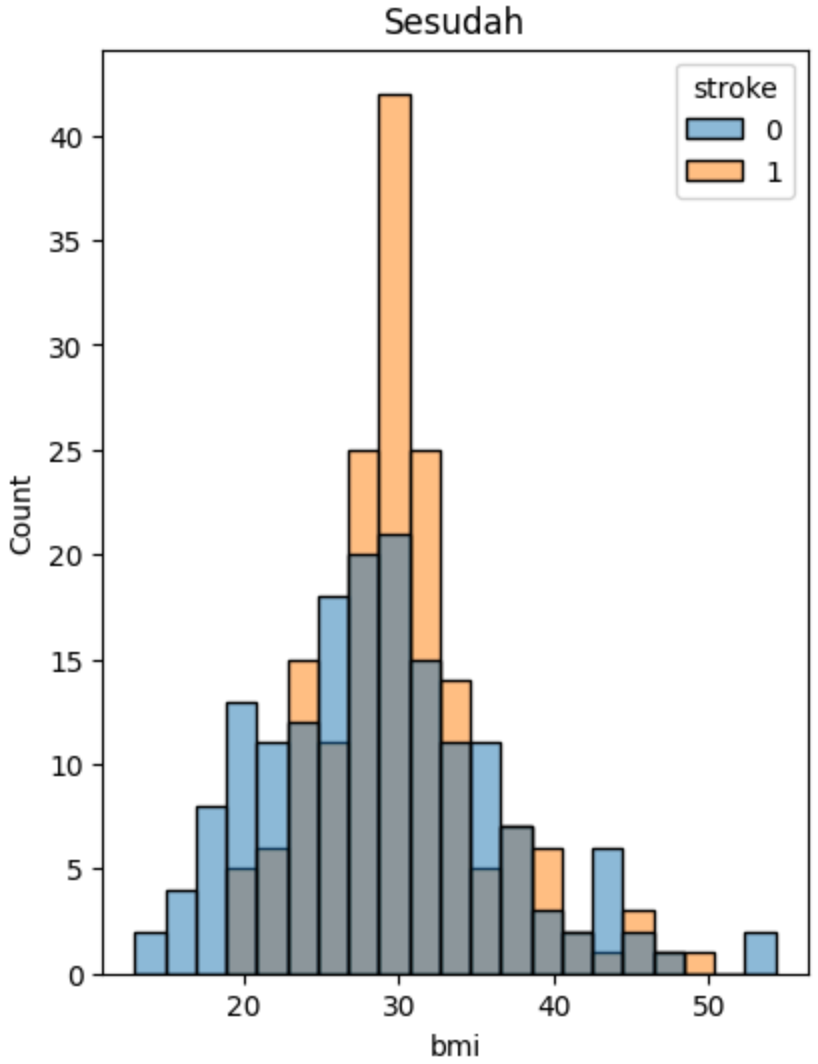
In [7]:
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

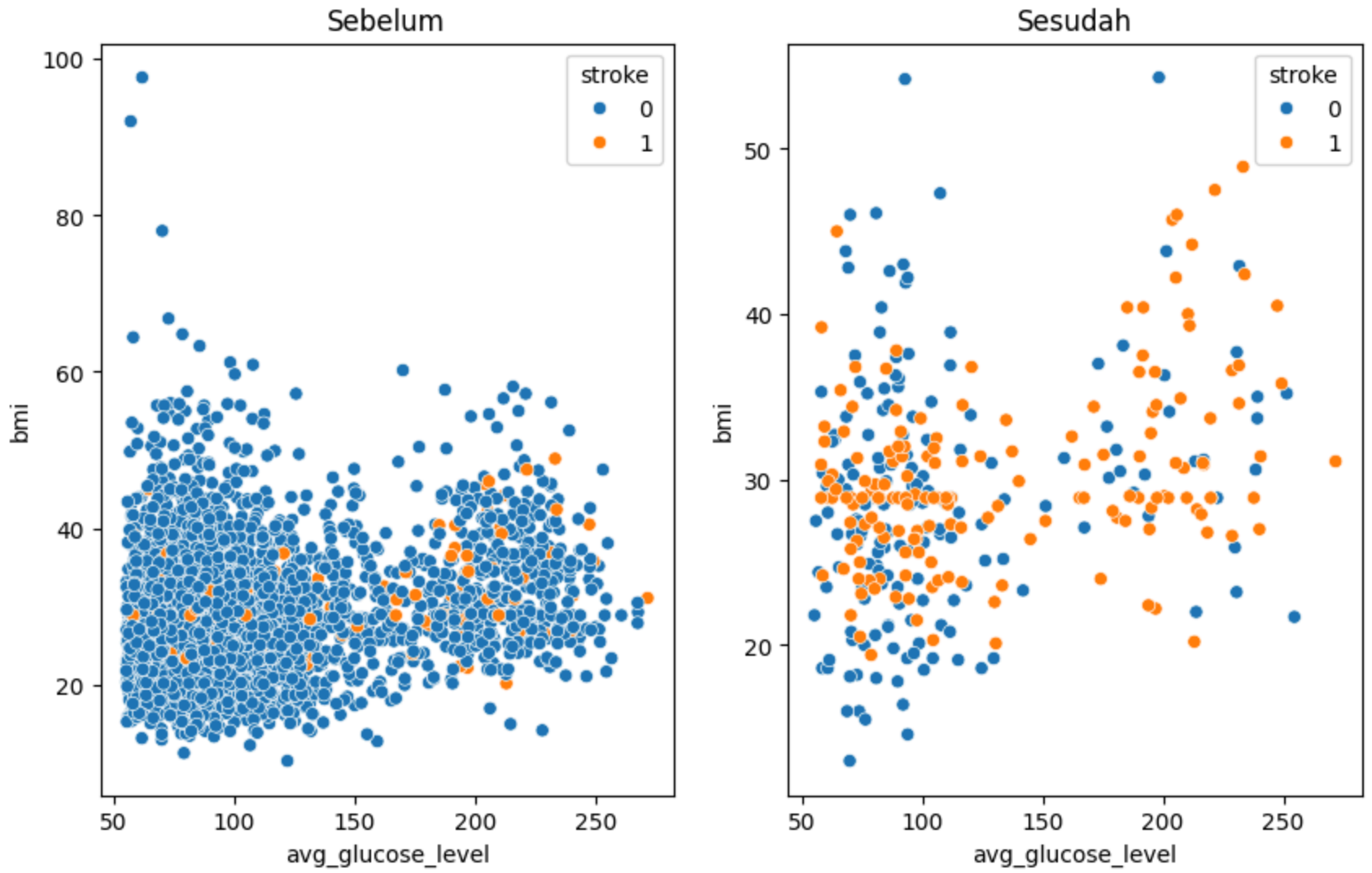
f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks(rotation='horizontal')
plt.show()
```


Sebelum
stroke
0
1
Count
bmi
0
50
100
150
200
250
300
350
20
40
60
80
100


Sesudah
stroke
0
1
Count
bmi
0
5
10
15
20
25
30
35
40
20
30
40
50

In [8]:
```
import seaborn as sns

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sebelum, hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sesudah, hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```



oversampling

In [9]:
```
from imblearn.over_sampling import RandomOverSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

over_sampling = RandomOverSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']
#imputation
df_X['bmi'].fillna(df_X['bmi'].mean(), inplace=True)

#categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25, random_state=42)

X_resampled, y_resampled = over_sampling.fit_resample(X_train, y_train)

print('sebelum random oversampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah random oversampling')
print(pd.DataFrame(y_resampled).value_counts())
```

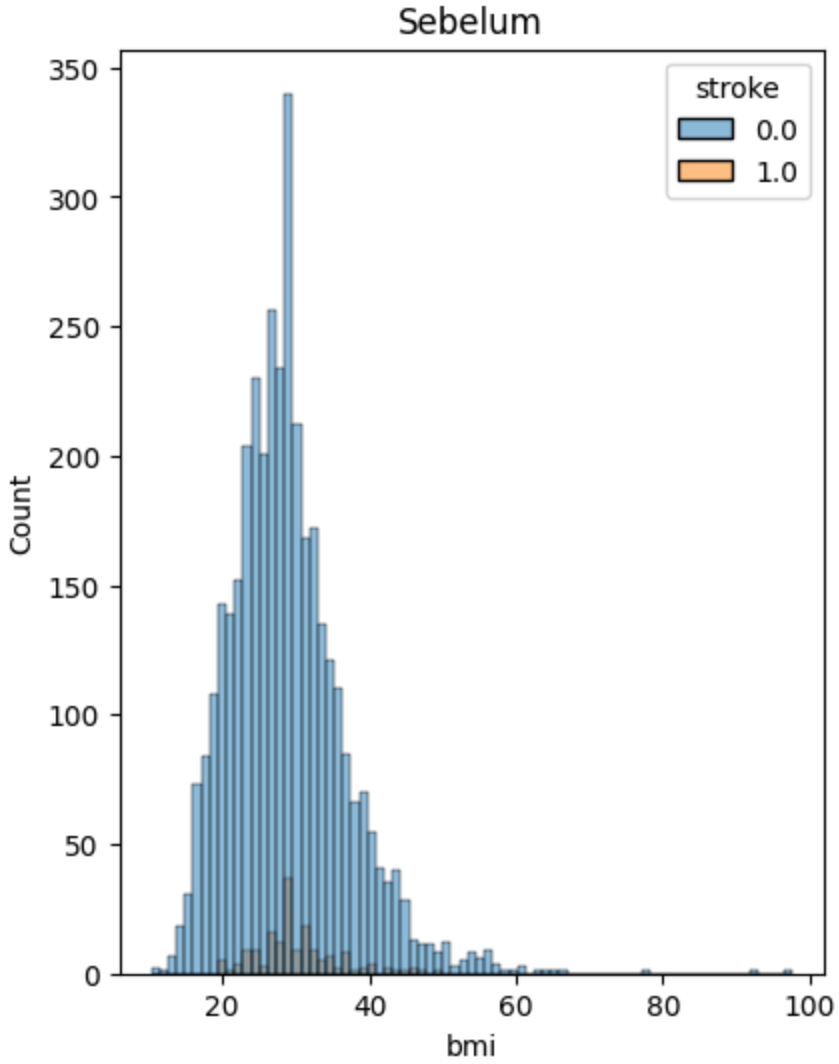
```
sebelum random oversampling
0.0    3663
1.0     169
Name: count, dtype: int64
setelah random oversampling
0.0    3663
1.0    3663
Name: count, dtype: int64
```

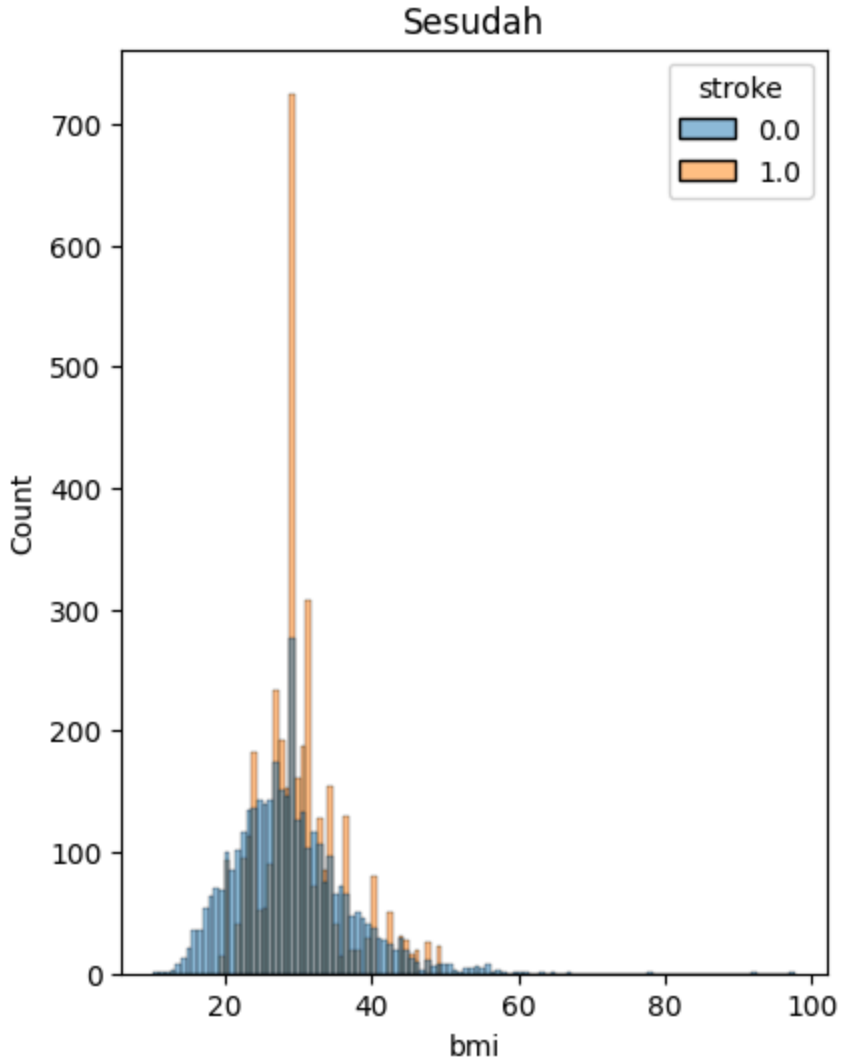
In [10]:
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

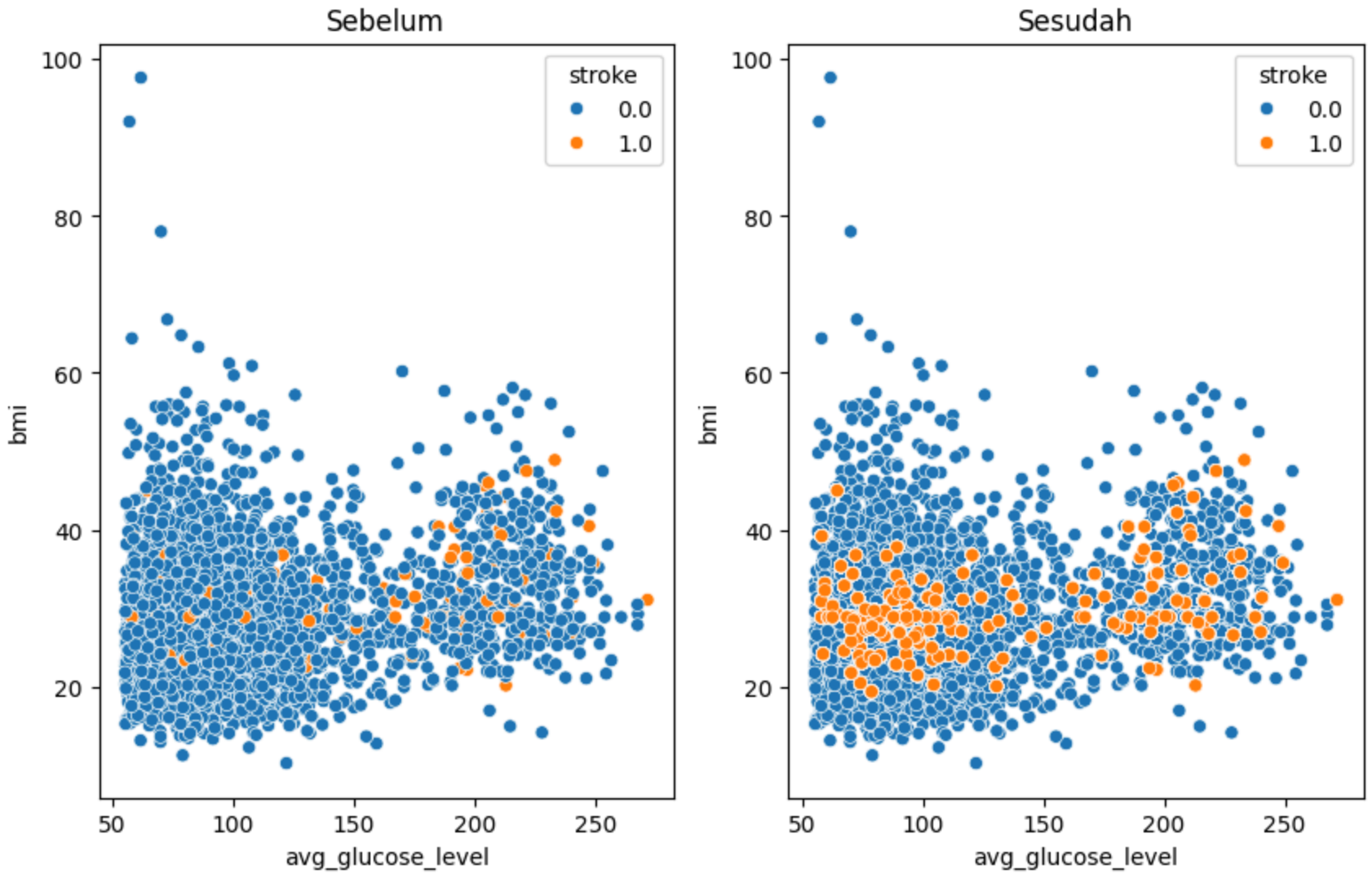
f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```


Sebelum
stroke
0.0
1.0
Count
bmi
0
50
100
150
200
250
300
350
20
40
60
80
100


Sesudah
stroke
0.0
1.0
Count
bmi
0
100
200
300
400
500
600
700
20
40
60
80
100

In [11]:
```
import seaborn as sns

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sebelum, hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sesudah, hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```



SMOTE

In [12]:
```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id','stroke'],axis=1)
df_y = df['stroke']
#imputation
df_X['bmi'].fillna(df_X['bmi'].mean(), inplace=True)
#categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
smote = SMOTE(random_state=0)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
print('Sebelum SMOTE')
print(pd.DataFrame(y_train).value_counts())
print('Setelah SMOTE')
print(pd.DataFrame(y_resampled).value_counts())
```

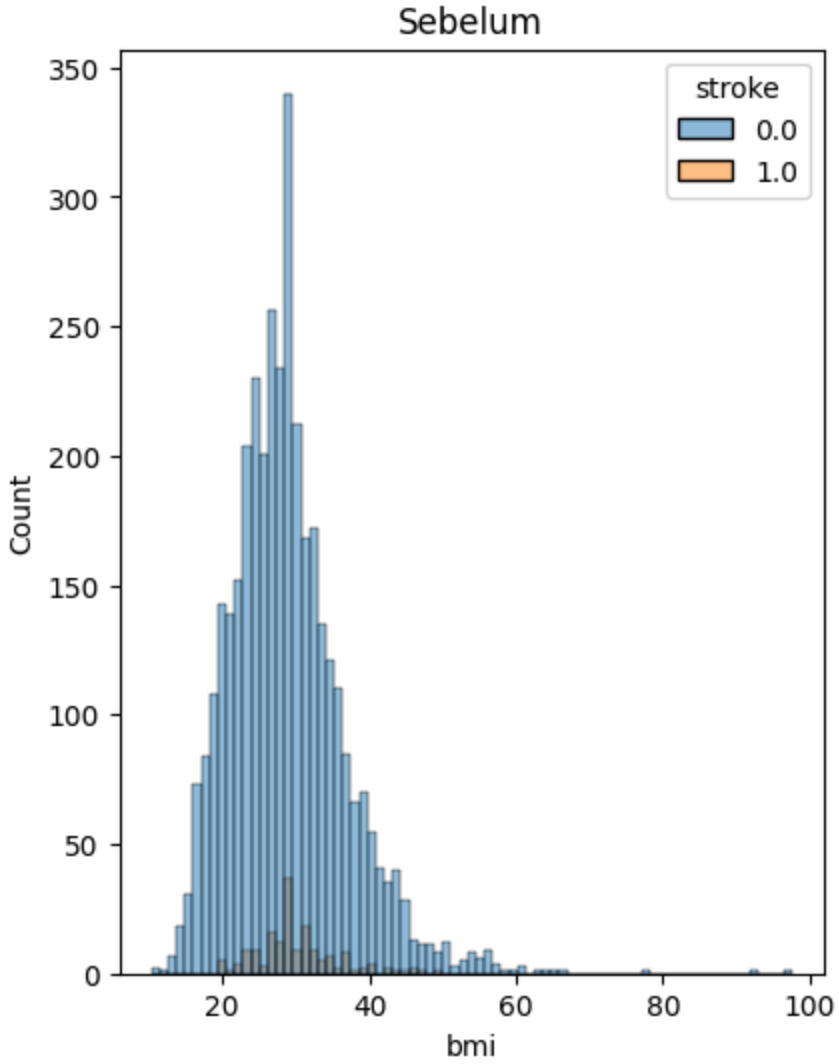
```
Sebelum SMOTE
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah SMOTE
0.0    3663
1.0    3663
Name: count, dtype: int64
```

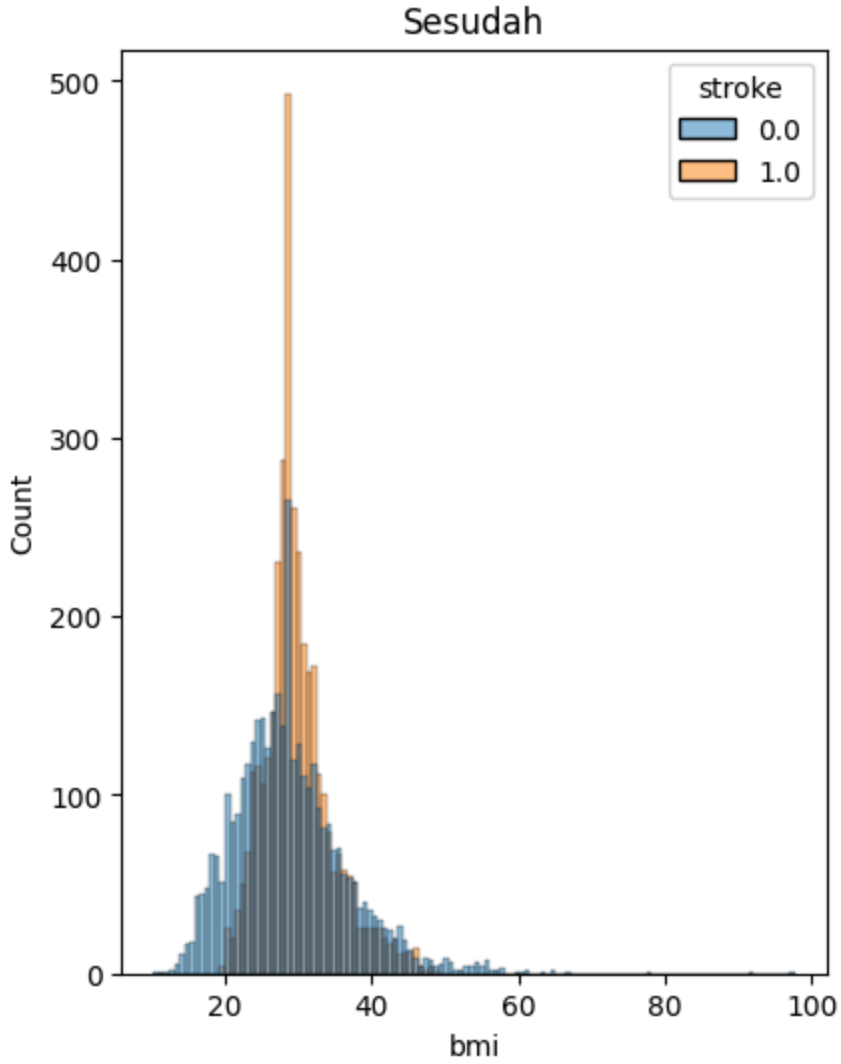
In [13]:
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

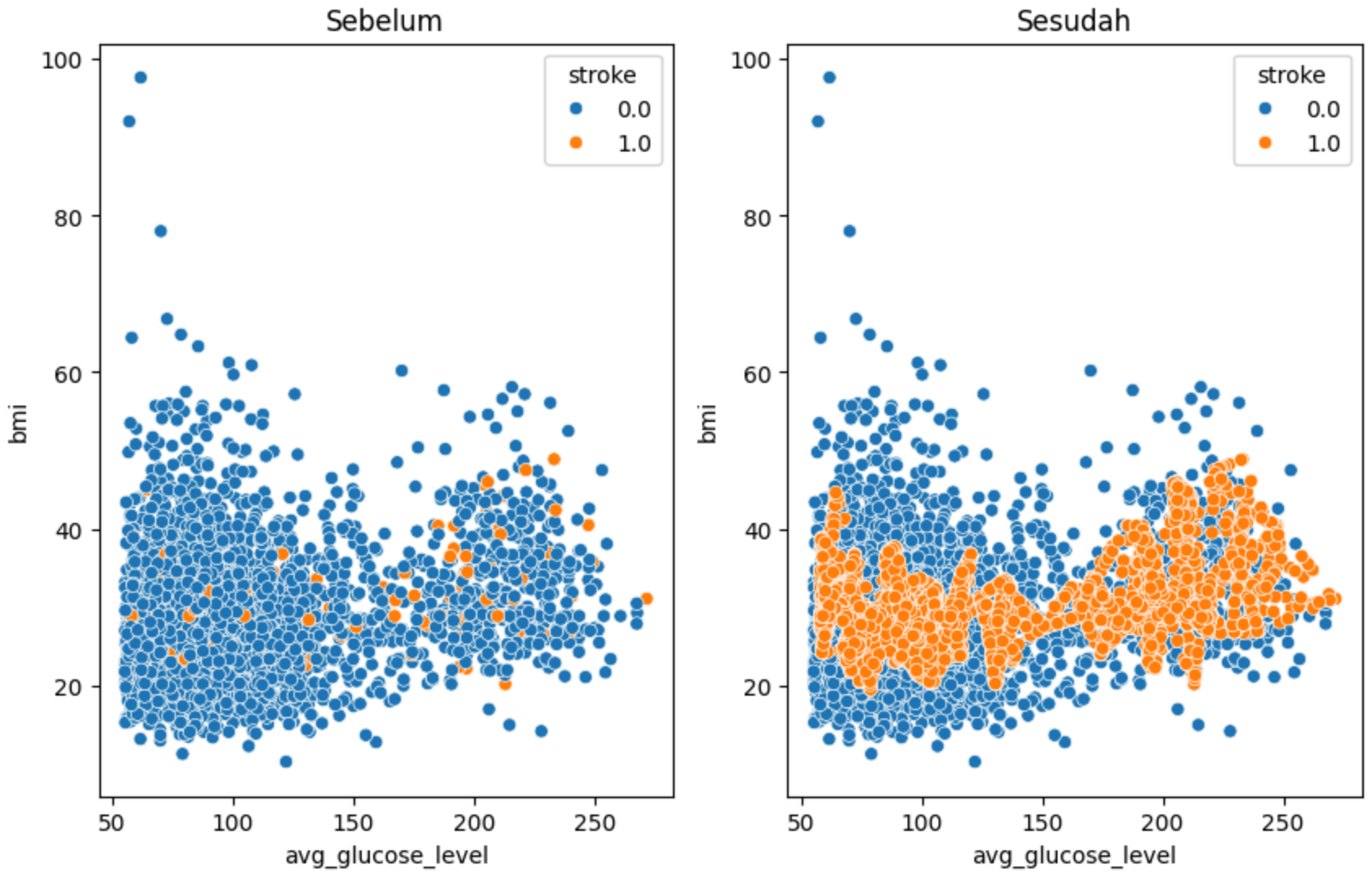
f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```


Sebelum
stroke
0.0
1.0
Count
bmi
0
50
100
150
200
250
300
350
20
40
60
80
100


Sesudah
stroke
0.0
1.0
Count
bmi
0
100
200
300
400
500
20
40
60
80
100

In [14]:
```
import seaborn as sns

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sebelum, hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sesudah, hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```



ADASYN

In [15]:
```
from imblearn.over_sampling import ADASYN
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id','stroke'],axis=1)
df_y = df['stroke']
#imputation
df_X['bmi'].fillna(df_X['bmi'].mean(), inplace=True)
#categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
adasyn = ADASYN(random_state=0)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)
print('Sebelum ADASYN')
print(pd.DataFrame(y_train).value_counts())
print('Setelah ADASYIN')
print(pd.DataFrame(y_resampled).value_counts())
```

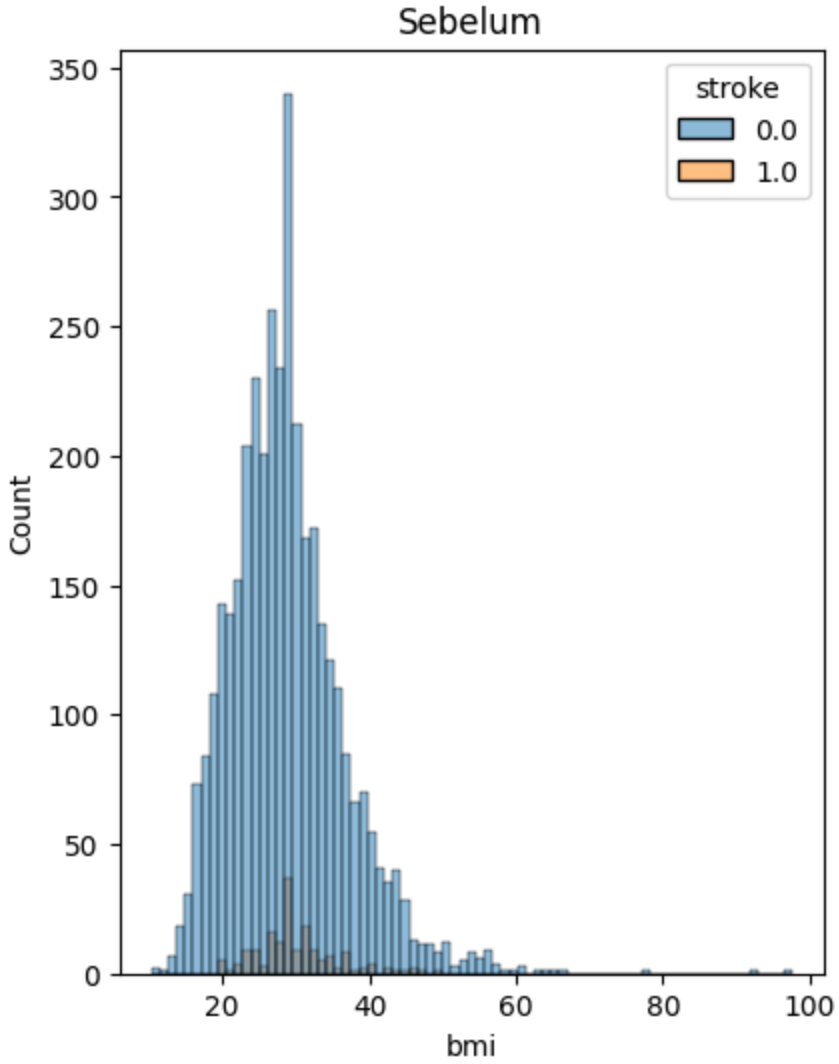
```
Sebelum ADASYN
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah ADASYIN
1.0    3676
0.0    3663
Name: count, dtype: int64
```

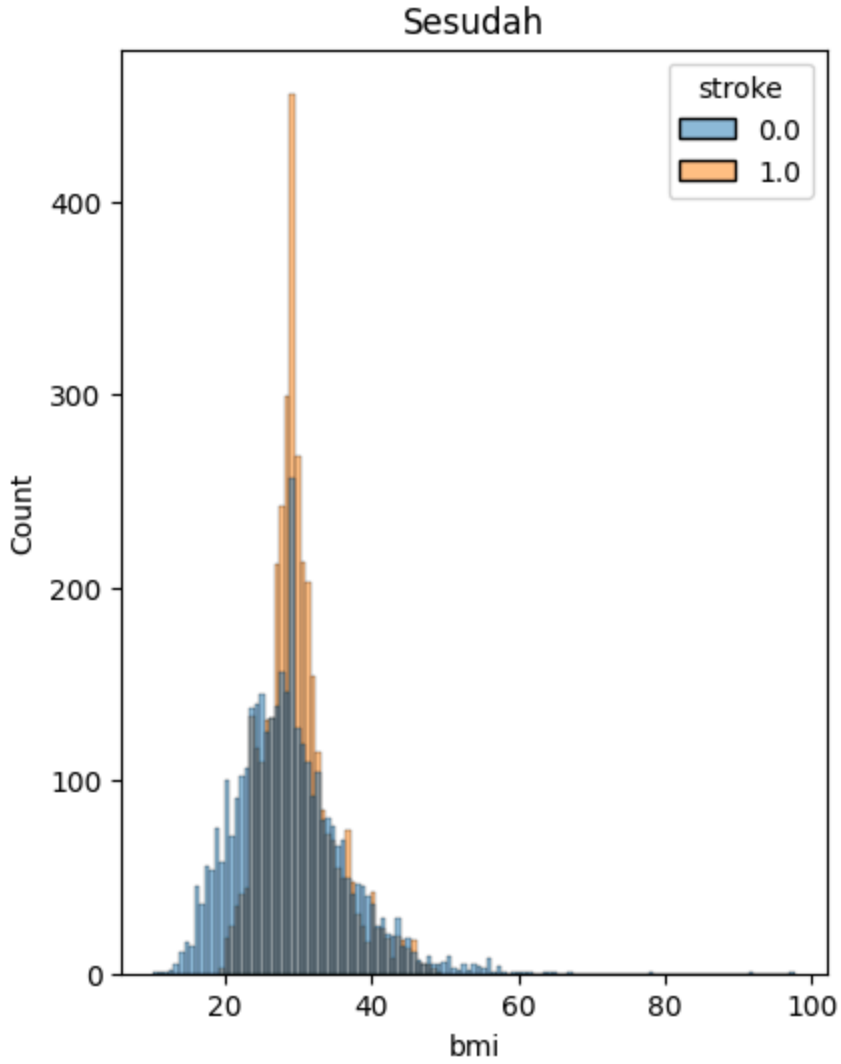
In [16]:
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

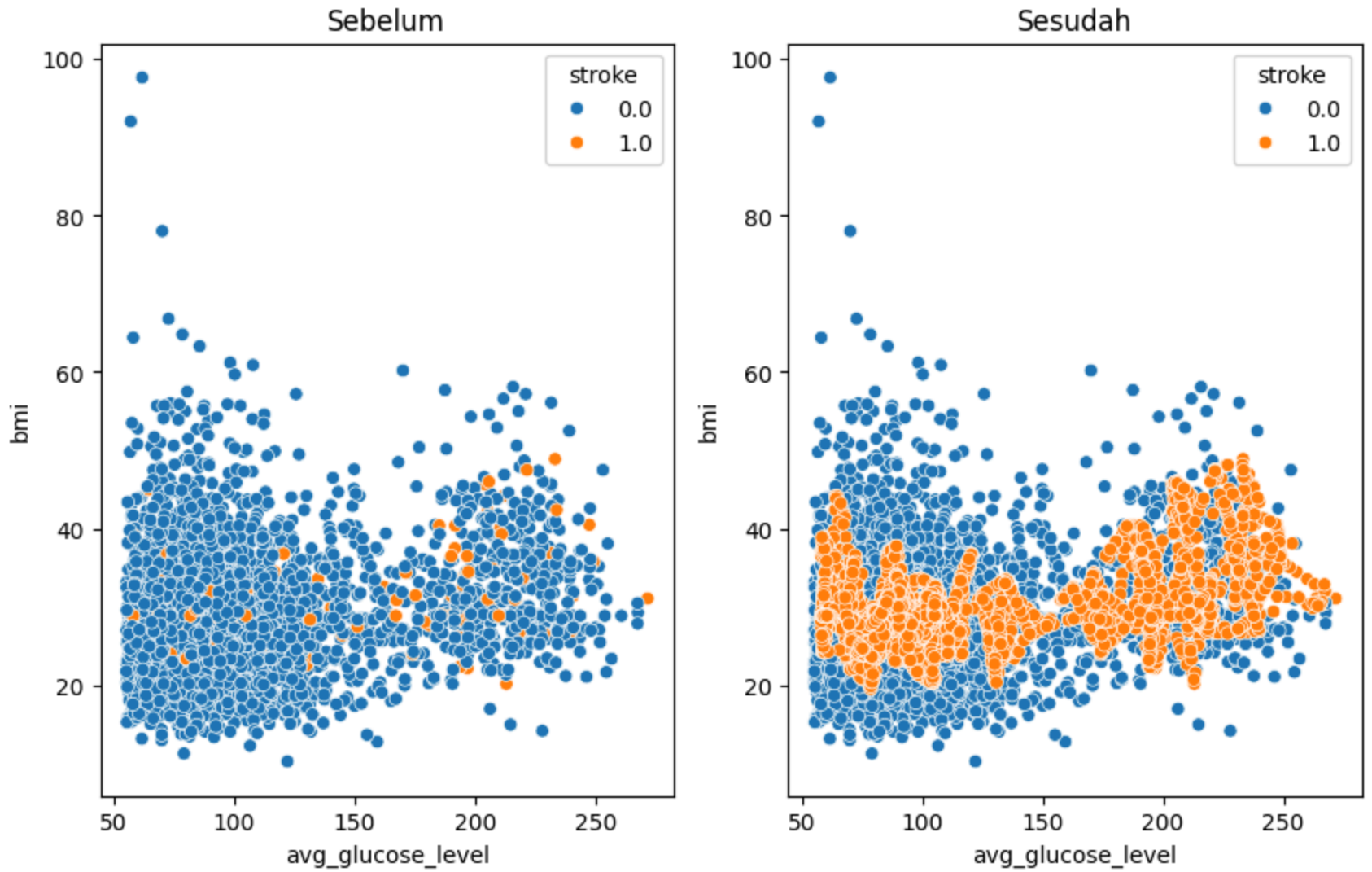
f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```


Sebelum
stroke
0.0
1.0
Count
bmi
0
50
100
150
200
250
300
350
20
40
60
80
100


Sesudah
stroke
0.0
1.0
Count
bmi
0
100
200
300
400
20
40
60
80
100

In [17]:
```
import seaborn as sns

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sebelum, hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sesudah, hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```



Tugas 2:

- Dengan menggunakan dataset lain, kombinasikan SMOTE atau ADASYN dengan classifier.

- Tampilkan performa model (accuracy, precision, recall) sebelum dan sesudah data balancing pada training set nya.